1. Start here

Match students and local participants using two spreadsheets

This tool suggests one-to-one matches for the programme. You edit two CSV files in Excel or another spreadsheet application, choose them in a local browser page, and download two output files. No Python installation, account or mathematical optimisation knowledge is needed with the Windows package.

Purpose	File	What it contains
Input	`config.csv`	Weights and matching settings.
Input	`problem.csv`	One row for each student or local participant.
Output	`result.csv`	One row for each suggested pair.
Output	`report.txt`	A brief explanation, counts and unmatched IDs.

First time on a Windows computer

1. Download `PerfectMatchJYU-Windows.zip` from the project's GitHub Releases page, or obtain it from your IT colleague. This is the ready-to-run package; GitHub's **Code / Download ZIP** contains source code instead.
2. Right-click the ZIP and choose **Extract All**. Keep the entire extracted folder together, including `_internal`.
3. In the extracted `PerfectMatch` folder, double-click `PerfectMatch.exe`. Your usual browser opens automatically. Python and the matching solver are already included.
4. The address normally starts with `http://localhost:8765`. "Localhost" means your own computer. Internet access is not needed for matching; CSV data stays on your computer.

Try the included example

Click **Try the included example**, then **Find matches**. When the page says **Matching complete**, read the brief report below the table. Click **Download result.csv** and **Download report.txt**. Your browser usually saves them in Downloads.

Student	Local	Score, rounded
S01	L01	0.724
S02	L02	0.662
S03	L03	0.794

All three students are matched. Local participant L04 remains unmatched. These are fictional example records. The unchanged template has 12 possible pairs and 8 eligible pairs.

For each real run: download the two templates, replace the fictional participants, save as CSV UTF-8, choose both files on the page, and click Find matches. Review and save both downloads. Keep the two inputs and two outputs in a dated folder. Use **Close app** when finished.

2. Prepare the participant list

Edit `problem.csv`

Keep the heading row unchanged. Delete the fictional participant rows before entering real data. Each person gets one row; students and locals are in the same file. There is no need to sort the rows.

IDs: use unique codes such as S001 and L001. Start with a letter or number; use only letters A–Z, numbers, underscores, hyphens or dots. The same code cannot be used for two people. Use **Text** format in Excel if codes contain leading zeroes. Keep names and contact details in your normal programme records, linked by ID.

Column heading	What to enter
`id`	A unique participant code, for example S001. Required.
`side`	Exactly `student` or `local`. Required.
`profile`	Profile tags, for example `a single person`.
`hobbies`	The person's interests: `hiking; cooking`.
`preferred_hobbies`	Interests appreciated in a friend: `hiking; sports`.
`gender`	Their stated gender, using consistent wording.
`preferred_gender`	Accepted genders separated by semicolons, or `no preference`.
`native_languages`	Native languages: `Finnish; Swedish`.
`languages`	Languages for daily life: `English; Finnish`.
`age`	A whole number from 1 to 120, or blank if unknown.
`preferred_age`	A range such as `22-35`, `no preference`, or blank.
`pets`	`none`, `has pets`, `avoid pets`, or `unknown`. See page 3.
`appreciate`	Qualities appreciated in a friend: `kind; reliable`.
`information`	Comparable tags such as `weekends; museums`; see page 3.
`degree`	Degree category, for example `Masters`. May be blank for locals.
`field`	Field of science, for example `Health Sciences`.
`must_match_1`	First required criterion, or blank. Allowed names are on page 3.
`must_match_1_values`	Accepted values for the first criterion, or blank with its criterion.
`must_match_2`	Optional alternative criterion.
`must_match_2_values`	Accepted values for the alternative, or blank with its criterion.

Save a CSV that the tool can read

Choose **Save As** and a **CSV UTF-8** format. Keep the `.csv` ending. Accept the application's notice about keeping CSV format; only cell values are needed. Close the file before running the tool.

Comma-, semicolon- and tab-separated UTF-8 files are supported, including Excel's UTF-8 marker. Put multiple items inside one cell with semicolons; let Excel add the necessary CSV quotes. Do not add or remove separator commas in a text editor. If the file opens in one column, use Excel's **Data / From Text/CSV** import and select the correct delimiter.

3. Make preferences clear

What the tool understands

Use consistent words and short tags

Text is compared using shared items, ignoring capital letters. `Hiking` and `hiking` match; `hiking` and `walking` do not. The tool does not translate, infer synonyms, or understand the meaning of a paragraph. Spaces around an item are ignored.

Use the same vocabulary across the list. Separate items with semicolons, for example `music; reading; board games`. The `information` column also uses this comparison: different sentences about similar interests may score zero. Shared tags are easier to compare. Blank scoring fields usually provide no similarity credit; a missing age starts with an age score of 0.5 before any applicable preference adjustment.

Required criteria: one passing alternative is enough

Use one of these exact names in `must_match_1` or `must_match_2`:

Criterion	Compared with the other person's column
`profile`	`profile`
`hobbies`	`hobbies`, not preferred hobbies
`gender`	`gender`
`languages`	`languages` (daily use), not native languages alone
`degree`	`degree`
`field`	`field`

Fill in a criterion and its values together. A person with no required criteria has all four must-match cells blank. A person with one criterion must have it satisfied. With two criteria, **at least one must pass**; the tool does not require both. Each side's requirements are checked separately.

Example: `must_match_1 = hobbies`, values `hiking; sports`; `must_match_2 = languages`, values `Finnish`. The partner must have hiking or sports among their hobbies, *or* Finnish among their daily-use languages. The partner's own required criteria must also pass.

Gender, pets and age

Gender preferences must work in both directions when the check is on. A blank preference or `no preference` allows any gender. A blank gender cannot satisfy a partner's specific gender preference.

For pets, `none` means no pets at home and no pet restriction recorded; `has pets` means pets are present; `avoid pets` means a partner without pets is needed; `unknown` means it needs checking. With the pets check on, `has pets` paired with `avoid pets` is rejected. Unknown or blank pet data does not reject a pair and is flagged in the report. This single category cannot describe complex or animal-specific situations; review those with participants.

Age and preferred age ranges affect the score. They do **not** exclude a pair. A required age limit cannot be expressed as a must-match criterion in this version.

4. Choose weights and settings

Edit only the value column in `config.csv`

Keep the `setting`, `value` and `description` headings and all setting rows. Higher weights give that topic more influence on the score. Zero removes its influence on scoring; it does not turn off a required criterion.

Setting	Default	Meaning
`weight_profile`	34	Similar profile tags.
`weight_hobbies`	23	Hobbies against the other's preferred hobbies.
`weight_appreciate`	10	Shared qualities appreciated in a friend.
`weight_languages`	9	Shared native and daily-use languages.
`weight_age`	9	Age similarity and preferred ranges.
`weight_info`	7	Shared information tags.
`weight_degree`	5	Shared degree categories.
`weight_field`	3	Shared fields of science.
`enforce_gender`	TRUE	Apply both gender preferences.
`enforce_pets`	TRUE	Reject pets/avoid-pets conflicts.
`enforce_must_match`	TRUE	Apply each person's required criteria.
`minimum_score`	0	Exclude pairs below this score (0 to 1).
`solver_time_limit_seconds`	60	Seconds per solving stage (1 to 86400).

Change priorities one step at a time

The default weights add up to 100, so profile contributes 34% of the overall score. They do not have to add up to 100: the tool scales them to shares of the total. Use non-negative numbers, with at least one weight above zero.

For example, to emphasise hobbies, change `weight_hobbies` from 23 to 30 and `weight_profile` from 34 to 27. Leave the other weights unchanged. Run again and compare both matches and the unmatched list. Decimal points or decimal commas are accepted in numeric setting cells; do not use thousands separators or percentage signs.

Preferences that must be respected

Leave the three `enforce_...` values at `TRUE` for ordinary use. `FALSE` switches that check off. The report names disabled checks. Use a disabled check only for a deliberate comparison and review the affected preferences before using its suggestions.

A higher minimum score can reduce the number of matches. At the default of 0, every pair passing the enabled checks remains eligible, including a pair scoring zero. Try changing weights before adding a score threshold. A score is not a success probability.

The time limit applies separately to three solving stages. A 60-second setting can allow about three minutes of solving, plus data processing. If the tool cannot confirm a completed solution in time, the run fails with a clear message instead of reporting an unfinished solution as successful.

5. Review the results

Open the report before confirming any pair

1. Check that the page says **Matching complete** and the report's run time is the run you just made. The time is in UTC and may differ from your local clock.
2. Check counts of students, locals, eligible pairs and selected matches. Unexpected counts can indicate an input or setting mistake.
3. Read the unmatched list and any review notes, including disabled checks or unknown pet information.
4. Download `result.csv` and review the suggested pairs against programme records. Discuss exceptions and practical needs before confirming matches.
5. Keep copies of both inputs and both outputs for a record of the decision.

What the result columns mean

`student_id` and `local_id` identify the pair. `score` is the overall weighted similarity, between 0 and 1. The eight columns ending in `_score` show the component similarities *before* weights are applied. Result values are rounded to six decimals; the report shows the minimum, average and maximum to three decimals. A score of 0.724 does not mean a 72.4% chance of a good friendship.

The programme first tries to match as many people as possible. Among solutions with that number of pairs, it improves the lowest pair score, then the total score. Each person appears in at most one selected pair. Someone may not receive their individually highest-scoring partner because partners cannot be shared. Equally good solutions may produce different pairs.

The report distinguishes people with no eligible partner from people whose eligible partners were allocated elsewhere. Its summary states whether every student is matched; if not, it shows an mbox-ATTENTION line and gives each unmatched student a reason: how many locals fail each check, or which students compete for too few eligible locals (for example, three students who can only be paired with the same two locals). Relax a rule for one of the students named, lower the minimum score, or add a suitable local, then run again. Pair-check failure counts can overlap: do not add them together as a total. With no selected pairs, the result CSV still has its headings and the report explains the situation.

If something goes wrong

Message or symptom	What to do
Missing or renamed headings	Restore the heading row from the matching example CSV. Keep every column, even if its cells are blank.
Duplicate ID or invalid value	Correct the indicated row. Use student/local, valid numbers and the listed criterion names.
Encoding or CSV error	Save again as CSV UTF-8 from the spreadsheet application.
Page cannot connect	Start `PerfectMatch.exe` again and use the newly opened page. Closing a tab alone does not stop the app.
No matches or too few	Check side counts, required criteria and wording; review the minimum score and unmatched reasons.
Solver could not finish	Increase the time setting; ask IT if it continues.
An error appears	Correct the indicated input and choose the updated file again. Earlier downloaded files remain on disk; check the report timestamp.

6. Setup and technical reference

For the colleague who prepares the computer

Recommended: ready-to-run Windows package

The package targets 64-bit Windows and includes Python, the libraries and CBC. Extract it into a writable folder; keep `PerfectMatch.exe` with its `_internal` folder. No Caddy, package installation or administrator rights are needed for normal use. Test with the included example. The executable is unsigned; if organisational policy blocks it, ask IT to review it.

The application listens only on this computer's loopback address, normally port 8765. If that port is busy, it chooses another and opens the correct page. Each CSV may be up to 4 MB in the browser. Inputs and solver files are temporary; results stay in the page until refreshed. Save the outputs before closing or reloading. Use **Close app** to stop the background process. Several open instances are independent.

Alternative: run from source

1. Install Python 3.12 from python.org, following IT policy. Enable **Add python.exe to PATH**. Extract the source ZIP into a writable folder.
2. Run `setup_windows.bat` once, with internet access. It creates `.venv`, installs the packages, and preserves existing files in `input`. Do not disable certificate checks if setup reports network errors.
3. Double-click `start_browser.bat` to open the local browser interface. Keep its command window open while working. macOS/Linux users can install `requirements.txt` in a Python environment and run `python browser_app.py`.

Command-line use (optional)

From the project folder after installation:

```
python run_matching.py --config examples/config.csv
--problem examples/problem.csv --result output/result.csv
--report output/report.txt
```

Type this as **one line**, using the installed environment's Python. On Windows:

```
.venv/Scripts/python.exe
```

The older `run_matching.bat` reads the two CSVs from `input` and writes results into `output`. It overwrites earlier outputs on success; a failed run may leave older files.

Verification and maintenance

The templates are synthetic. Do not add real participant files to Git. The report records SHA-256 identifiers of the processed input files; the browser normalises UTF-8 encoding when passing them to the solver. Keep the original inputs too.

Run checks with:

```
python -m unittest discover -s tests -v
```

Checks cover CSV validation, matching priorities and the local HTTP interface. To build the Windows package, see `build_windows.ps1` and the README. PuLP 3.3.0 supplies CBC. The lowest-score priority uses a tolerance of 10^{-7}.

This self-contained A4 source, `manual.tex`, uses no external images. Compile with `pdflatex manual.tex` twice or `tectonic manual.tex`. The repository also includes `manual.pdf`.